

Sentiment API (lot 1, miniature) : démarche de test

Cours Développement et déploiement d'applications intelligentes, semaine 1

A. Larhlimi

Cette fiche décrit, dans l'ordre, la démarche pour préparer le poste, configurer l'application miniature, puis la tester : tests automatisés, API en conditions réelles et interface Streamlit. Le point de départ est l'archive `lot1-code-fil-rouge-miniature.zip`.

1. Préparation du poste

- Installer Python 3.11 (ou plus récent) avec `venv` et `pip`, Git, et un éditeur (VS Code).
- Décompresser l'archive, puis se placer dans le dossier :

```
cd sentiment-app
```

- Si le dossier doit devenir le dépôt du binôme : `git init`, puis chaque membre fait au moins un commit (le test `test_git_history.py` l'exige).

2. Environnement virtuel

```
python -m venv .venv
source .venv/bin/activate          # Windows : .venv\Scripts\activate
```

Le préfixe `(.venv)` doit apparaître dans l'invite. Sans lui, `test_virtualenv_active` échouera.

3. Dépendances

```
pip install --upgrade pip
pip install -r requirements.txt
pip freeze > requirements.lock
```

Raccourci : `scripts/setup.sh` (Linux/macOS) ou `powershell -ExecutionPolicy Bypass -File scripts\setup.ps1` (Windows) enchaîne les étapes 2, 3 et 4.

4. Configuration

```
cp .env.example .env          # Windows : copy .env.example .env
```

Ouvrir `.env` et remplacer `API_KEY=changez-moi` par une valeur choisie. Laisser `API_URL=http://127.0.0.1:8000` et `MODELE=models/sentiment.joblib`. Ce fichier n'est jamais commité (il est dans `.gitignore`).

5. Diagnostic du poste

```
python scripts/check_setup.py
```

Attendu : toutes les lignes en OK sauf « modèle entraîné », donc 1 erreur(s) à ce stade. Après l'étape 6, relancer la commande pour obtenir 0 erreur(s).

6. Entraînement du modèle

```
python train.py
```

Attendu : Modèle enregistré : `models/sentiment.joblib` et l'accuracy en validation croisée. Deux fichiers apparaissent dans `models/` : `sentiment.joblib` et `metrics.json`.

7. Tests automatisés

```
pytest -q
```

Attendu : 11 passed (ou 10 passed, 1 skipped si le dépôt Git n'a pas encore deux auteurs). Pour cibler un fichier : `pytest -q tests/test_api.py`, qui teste l'API sans serveur grâce à `TestClient`.

8. Test de l'API en conditions réelles

Terminal 1 (venv actif) :

```
uvicorn app.main:app --reload
```

Ouvrir `http://127.0.0.1:8000/docs` dans le navigateur, puis vérifier :

- GET `/health` renvoie `{"status": "ok", "modele_charge": true}` ;
- POST `/predict` avec `{"texte": "Un film magnifique, j'ai adoré."}` renvoie label, confiance et version_model ;
- POST `/predict` avec `{"texte": ""}` renvoie 422 (validation Pydantic).

Variante en ligne de commande :

```
curl -X POST http://127.0.0.1:8000/predict -H "Content-Type: application/json" -d '{"texte": "Ennuyeux et nul."}'
```

Test du cas d'erreur : arrêter le serveur, renommer temporairement `models/sentiment.joblib`, relancer uvicorn. `/health` renvoie alors `modele_charge: false` et `/predict` renvoie 503 Modèle non chargé. Remettre le fichier ensuite.

9. Test sur Streamlit

Terminal 2 (venv activé, l'API de l'étape 8 toujours en marche) :

```
streamlit run ui/app.py
```

Le navigateur s'ouvre sur `http://localhost:8501`. Vérifier dans l'ordre :

1. Barre latérale : « API en ligne ». Si elle affiche « API hors ligne », l'API n'est pas lancée ou `API_URL` est faux dans `.env`.
2. Onglet « Un avis » : saisir un texte positif, cliquer sur **Analyser**, lire le label, la confiance et la version du modèle. Refaire avec un texte négatif.
3. Onglet « Un fichier CSV » : téléverser `data/sample_reviews.csv`. Le tableau affiche les colonnes `label_predit` et `confiance`, suivi d'un graphique en barres des classes.
4. Test négatif : arrêter uvicorn (Ctrl+C dans le terminal 1), recharger la page Streamlit ; la barre latérale passe à « API hors ligne » et le bouton **Analyser** affiche une erreur d'appel. Relancer l'API pour terminer.

10. Livraison

```
git add -A
git commit -m "TP1 : environnement et Sentiment API miniature"
git push
```

Vérifier que `.venv/`, `.env` et `models/*.joblib` n'apparaissent pas dans le commit, puis que le workflow GitHub Actions (`ci.yml`) passe au vert : il réinstalle les dépendances, entraîne le modèle et exécute `pytest`.

Pièges fréquents

Symptôme	Cause probable	Correction
503 Modèle non chargé	<code>python train.py</code> non lancé, ou <code>MODELE</code> mal renseigné	Entraîner le modèle, vérifier <code>.env</code>
« API hors ligne » dans Streamlit	uvicorn arrêté, ou port différent de <code>API_URL</code>	Relancer l'API, aligner <code>API_URL</code>
<code>test_virtualenv_active</code> en échec	venv non activé	Activer <code>.venv</code> (sur Colab : <code>--deselect tests/test_environment.py::test_virtualenv_active</code>)
Dépôt de plusieurs centaines de Mo	<code>.venv/</code> commité	Vérifier <code>.gitignore</code> avant le premier <code>git add</code>

Récapitulatif des commandes

```
cd sentiment-app
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env
python scripts/check_setup.py
python train.py
pytest -q
uvicorn app.main:app --reload          # terminal 1
streamlit run ui/app.py                 # terminal 2
```